

দিন ৭

রিভিশন ও Mock Interview

আজ নতুন কিছু না — যা শিখেছো সেটা গুছিয়ে, সময় ধরে, মুখে বলে প্র্যাকটিস করো

আজ যা শিখবো

- প্রশ্ন দেখে প্যাটার্ন চেনার ম্যাপ
- Top 12 Must-Do প্রশ্ন — এক নজরে কোড
- Complexity মাস্টার টেবিল
- ৩টা সম্পূর্ণ Mock Interview (সংলাপসহ)
- ৫০টা র‍্যাপিড-ফায়ার থিওরি প্রশ্ন
- PHP ইন্টারভিউ চিট শিট ও শেষ মুহূর্তের চেকলিস্ট

কোর্স ভিডিও: সব (১-৪০) — রিভিশন

দিন শেষে তুমি পারবে

- যেকোনো প্রশ্ন দেখে ২ মিনিটে সঠিক প্যাটার্ন চিনতে পারবে
- ৬-ধাপের ফর্মুলা মেনে মুখে বলে বলে সলভ করতে পারবে
- আত্মবিশ্বাস নিয়ে ইন্টারভিউতে ঢুকতে পারবে

স্ট্যান্ডি শিডিউল (প্রায় ৫-৬ ঘণ্টা)

১ ঘণ্টা	প্যাটার্ন ম্যাপ + Top 12 কোড রিভিশন (খাতায় লিখে)
১.৫ ঘণ্টা	৩টা Mock পড়ো, তারপর নিজে টাইমার ধরে সলভ করো
১ ঘণ্টা	Timed Mock Set: ৩ রাউন্ড (নিচে)
৪৫ মিনিট	র‍্যাপিড-ফায়ার থিওরি জোরে জোরে
৩০ মিনিট	চিট শিট + চেকলিস্ট, তারপর বিশ্রাম!

অধ্যায় ১: প্রশ্ন দেখে প্যাটার্ন চেনো

ইন্টারভিউয়ের সবচেয়ে বড় দক্ষতা হলো প্রশ্ন পড়ে বুঝে ফেলা **কোন কৌশল** লাগবে। নিচের টেবিলটা মুখস্থ না করে, প্রতিটা সারির জন্য একটা করে প্রশ্ন মনে করো যেটা তুমি সলভ করেছো।

প্রশ্নে যে শব্দ/সংকেত দেখাবে	প্যাটার্ন	যে প্রশ্নগুলো করেছে	দিন
"জোড়া খোঁজো", "আগে দেখেছি কি না", "কতবার"	Hash Map / Set	Two Sum, Contains Duplicate, Anagram	১, ৩
"sorted array", "in-place", "O(1) space"	Two Pointer	Reverse, Move Zeroes, Remove Duplicates	১
"subarray/substring", "সবচেয়ে লম্বা/ছোট", "পাশাপাশি"	Sliding Window	Longest Substring Without Repeating	৩
"sorted" + "খোঁজো", "O(log n)"	Binary Search	Binary Search, Search Insert Position	২
"linked list", "মারখান", "cycle"	Fast-Slow Pointer	Middle, Linked List Cycle	৪
"bracket", "সবশেষে খোলা আগে বন্ধ", "undo"	Stack	Valid Parentheses	৪
"লেভেল ধরে", "সবচেয়ে ছোট পথ", "ধাপে ধাপে ছড়ায়"	BFS (Queue)	Level Order, Bipartite	৬
"সব পথ", "লাগোয়া অংশ", "গ্রিড"	DFS	Number of Islands, Components	৬
"কয়টা উপায়", "সর্বোচ্চ/সর্বনিম্ন", প্রতিটা ধাপে পছন্দ	Dynamic Programming	Climbing Stairs, House Robber, Coin Change	৫
"interval", "মিটিং", "সবচেয়ে বেশি কয়টা"	Sort + Greedy	Merge Intervals	৫
"tree", "depth", "subtree"	Recursion (DFS)	Max Depth, Validate BST	৬

কোনো প্যাটার্ন মনে না এলে?

1. Brute force বলো — সবসময় একটা আছে।
2. জিজ্ঞেস করো: "কোথায় একই কাজ বারবার হচ্ছে?" → সেটা মনে রাখতে hash map বা DP।
3. জিজ্ঞেস করো: "sort করলে কি সহজ হয়?" → two pointer / binary search / greedy।
4. ছোট উদাহরণ হাতে সলভ করো, নিজের ধাপগুলো লক্ষ করো — সেগুলোই অ্যালগরিদম।

Input সাইজ থেকে কার্যক্ষমত Complexity অনুমান

n-এর সীমা	গ্রহণযোগ্য Complexity	সম্ভাব্য কৌশল
$n \leq 20$	$O(2^n)$	Recursion / সব subset
$n \leq 1,000$	$O(n^2)$	Nested লুপ, সহজ DP
$n \leq 10^5 - 10^6$	$O(n)$ বা $O(n \log n)$	Hash map, two pointer, sort, sliding window
$n > 10^8$ বা আরও বড়	$O(\log n)$ বা $O(1)$	Binary search, গণিত

অধ্যায় ২: Top 12 Must-Do — এক নজরে কোড

প্রতিটা কোড খাতায় না দেখে লিখে মিলিয়ে নাও। যেটা আটকাবে, সেই দিনের PDF-এ ফিরে যাও।

১. Two Sum (দিন ১) — $O(n)$ / $O(n)$

PHP

```
function twoSum(array $nums, int $target): array {
    $seen = [];
    foreach ($nums as $i => $num) {
        if (isset($seen[$target - $num])) return [$seen[$target - $num], $i];
        $seen[$num] = $i;
    }
    return [];
}
echo json_encode(twoSum([2, 7, 11, 15], 9)); // [0,1]
```

২. Best Time to Buy/Sell Stock (দিন ১) — $O(n)$ / $O(1)$

PHP

```
function maxProfit(array $prices): int {
    $min = PHP_INT_MAX; $best = 0;
    foreach ($prices as $p) { $min = min($min, $p); $best = max($best, $p - $min); }
    return $best;
}
echo maxProfit([7, 1, 5, 3, 6, 4]); // 5
```

৩. Move Zeroes (দিন ১) — $O(n)$ / $O(1)$

PHP

```
function moveZeroes(array &$nums): void {
    $slow = 0;
    for ($fast = 0; $fast < count($nums); $fast++) {
        if ($nums[$fast] !== 0) { [$nums[$slow], $nums[$fast]] = [$nums[$fast], $nums[$slow]]; $slow++; }
    }
}
$a = [0, 1, 0, 3, 12]; moveZeroes($a); echo implode(",", $a); // 1,3,12,0,0
```

8. Binary Search (দিন ২) — $O(\log n)$ / $O(1)$

PHP

```
function binarySearch(array $a, int $t): int {
    $lo = 0; $hi = count($a) - 1;
    while ($lo <= $hi) {
        $mid = $lo + intval($hi - $lo, 2);
        if ($a[$mid] === $t) return $mid;
        if ($a[$mid] < $t) $lo = $mid + 1; else $hi = $mid - 1;
    }
    return -1;
}
echo binarySearch([-1, 0, 3, 5, 9, 12], 9); // 4
```

৫. Merge Sort (দিন ২) — $O(n \log n)$ / $O(n)$

PHP

```
function mergeSort(array $a): array {
    if (count($a) <= 1) return $a;
    $mid = intval(count($a), 2);
    $l = mergeSort(array_slice($a, 0, $mid)); $r = mergeSort(array_slice($a, $mid));
    $res = []; $i = $j = 0;
    while ($i < count($l) && $j < count($r)) $res[] = ($l[$i] <= $r[$j]) ? $l[$i++] : $r[$j++];
    return array_merge($res, array_slice($l, $i), array_slice($r, $j));
}
echo implode(",", mergeSort([5, 2, 9, 1, 3])); // 1,2,3,5,9
```

৬. Valid Anagram (দিন ৩) — $O(n)$ / $O(1)$

PHP

```
function isAnagram(string $s, string $t): bool {
    if (strlen($s) !== strlen($t)) return false;
    $c = [];
    for ($i = 0; $i < strlen($s); $i++) { $c[$s[$i]] = ($c[$s[$i]] ?? 0) + 1; $c[$t[$i]] = ($c[$t[$i]] ?? 0) - 1; }
    foreach ($c as $v) if ($v !== 0) return false;
    return true;
}
var_dump(isAnagram("listen", "silent")); // true
```

৭. Longest Substring Without Repeating (দিন ৩) — $O(n)$ / $O(k)$

PHP

```
function longest(string $s): int {
    $last = []; $left = 0; $best = 0;
    for ($r = 0; $r < strlen($s); $r++) {
        if (isset($last[$s[$r]]) && $last[$s[$r]] >= $left) $left = $last[$s[$r]] + 1;
        $last[$s[$r]] = $r;
        $best = max($best, $r - $left + 1);
    }
    return $best;
}
echo longest("abcabcbb"); // 3
```

৮. Reverse Linked List (দিন ৪) — $O(n)$ / $O(1)$

PHP

```
function reverseList(?ListNode $head): ?ListNode {
    $prev = null;
    while ($head !== null) { $next = $head->next; $head->next = $prev; $prev = $head; $head = $next; }
    return $prev;
}
echo listToString(reverseList(buildList([1, 2, 3]))); // 3 -> 2 -> 1 -> NULL
```

৯. Valid Parentheses (দিন ৪) — $O(n)$ / $O(n)$

PHP

```
function isValid(string $s): bool {
    $pair = [')' => '(', ']' => '[', '}' => '{']; $st = [];
    foreach (str_split($s) as $ch) {
        if (isset($pair[$ch])) { if (array_pop($st) !== $pair[$ch]) return false; }
        else $st[] = $ch;
    }
    return empty($st);
}
var_dump(isValid("{[()]}" )); // true
```

১০. Climbing Stairs / Fibonacci DP (দিন ৫) — $O(n)$ / $O(1)$

PHP

```
function climbStairs(int $n): int {
    $a = 1; $b = 1; // ways(0), ways(1)
    for ($i = 2; $i <= $n; $i++) [$a, $b] = [$b, $a + $b];
    return $b;
}
echo climbStairs(5); // 8
```

১১. Max Depth + Level Order (দিন ৬) — $O(n)$ / $O(n)$

PHP

```
function maxDepth(?TreeNode $r): int {
    return $r === null ? 0 : 1 + max(maxDepth($r->left), maxDepth($r->right));
}

function levelOrder(?TreeNode $root): array {
    $res = []; if (!$root) return $res;
    $q = new SplQueue(); $q->enqueue($root);
    while (!$q->isEmpty()) {
        $lvl = [];
        for ($i = count($q); $i > 0; $i--) {
            $n = $q->dequeue(); $lvl[] = $n->val;
            if ($n->left) $q->enqueue($n->left);
            if ($n->right) $q->enqueue($n->right);
        }
        $res[] = $lvl;
    }
    return $res;
}

$t = buildTree([3, 9, 20, null, null, 15, 7]);
echo maxDepth($t) . " " . json_encode(levelOrder($t)); // 3 [[3],[9,20],[15,7]]
```

১২. Number of Islands (দিন ৬) — $O(R \times C)$ / $O(R \times C)$

PHP

```
function numIslands(array $g): int {
    $cnt = 0;
    $sink = function (int $r, int $c) use (&$g, &$sink) {
        if ($r < 0 || $c < 0 || $r >= count($g) || $c >= count($g[0]) || $g[$r][$c] !== '1')
            return;
        $g[$r][$c] = '0';
        $sink($r + 1, $c); $sink($r - 1, $c); $sink($r, $c + 1); $sink($r, $c - 1);
    };
    for ($r = 0; $r < count($g); $r++)
        for ($c = 0; $c < count($g[0]); $c++)
            if ($g[$r][$c] === '1') { $cnt++; $sink($r, $c); }
    return $cnt;
}

echo numIslands([['1','1','0'], ['0','1','0'], ['0','0','1']]); // 2
```

অধ্যায় ৩: Complexity মাস্টার টেবিল

Data Structure অপারেশন

Data Structure	Access	Search	Insert	Delete	PHP-তে
Array	$O(1)$	$O(n)$	$O(n)$ (শেষে $O(1)$)	$O(n)$ (শেষে $O(1)$)	array
Linked List	$O(n)$	$O(n)$	$O(1)$ (head-এ)	$O(1)$ (node জানা থাকলে)	নিজে class / <code>SplDoublyLinkedList</code>
Stack	$O(n)$	$O(n)$	$O(1)$ push	$O(1)$ pop	array + <code>array_pop</code> / <code>SplStack</code>
Queue	$O(n)$	$O(n)$	$O(1)$ enqueue	$O(1)$ dequeue	<code>SplQueue</code>
Hash Map / Set	—	$O(1)$ গড়	$O(1)$ গড়	$O(1)$ গড়	associative array + <code>isset</code>
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	নিজে class
BST (skewed)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	—

Algorithm

Algorithm	Time	Space	মনে রাখো
Linear Search	$O(n)$	$O(1)$	যেকোনো array
Binary Search	$O(\log n)$	$O(1)$	শুধু sorted
Bubble / Selection Sort	$O(n^2)$	$O(1)$	Bubble stable, best $O(n)$
Merge Sort	$O(n \log n)$	$O(n)$	Stable, সবসময় $n \log n$
Quick Sort	$O(n \log n)$ গড়, $O(n^2)$ worst	$O(\log n)$	In-place, random pivot
Counting Sort	$O(n + k)$	$O(k)$	ছোট রেঞ্জ integer
Two Pointer / Sliding Window	$O(n)$	$O(1)$ / $O(k)$	Nested লুপের বদলে
Tree Traversal (DFS/BFS)	$O(n)$	$O(h)$ / $O(w)$	h = উচ্চতা, w = চওড়া
Graph BFS / DFS	$O(V + E)$	$O(V)$	visited লাগবেই
Recursive Fibonacci	$O(2^n)$	$O(n)$	Memo দিয়ে $O(n)$
DP (1D)	সাধারণত $O(n)$	$O(n) \rightarrow$ প্রায়ই $O(1)$	state, transition, base

অধ্যায় 8: Mock Interview ১ — Two Sum II (Sorted Array)

কীভাবে ব্যবহার করবে

প্রথমে শুধু প্রশ্নটা পড়ে ২০ মিনিটের টাইমার চালাও, নিজে জোরে জোরে বলে সলভ করো। তারপর সংলাপটা পড়ে মিলিয়ে দেখো কী বাদ পড়েছিল।

প্রশ্ন (ইন্টারভিউয়ার বলছেন)

একটা ascending sorted integer array আর একটা target দেওয়া আছে। এমন দুইটা সংখ্যার index খুঁজে দাও যাদের যোগফল target।

ইন্টারভিউয়ার এই প্রশ্নটা করো। সময় ২০ মিনিট।

তুমি ঠিক আছে। কয়েকটা জিনিস নিশ্চিত হয়ে নিই — array কি সবসময় sorted? নেগেটিভ সংখ্যা থাকতে পারে? আর ঠিক একটাই উত্তর আছে, নাকি না-ও থাকতে পারে?

ইন্টারভিউয়ার Sorted, নেগেটিভ থাকতে পারে, আর ধরে নাও ঠিক একটা উত্তর আছে। একই element দুইবার নেওয়া যাবে না।

তুমি একটা উদাহরণ নিই: [2, 7, 11, 15], target 9 — উত্তর index 0 আর 1। সবচেয়ে সহজ উপায় সব জোড়া চেক করা, $O(n^2)$ । এরপর Two Sum-এর hash map দিয়ে $O(n)$ time, $O(n)$ space করা যায়।

ইন্টারভিউয়ার ভালো। আর কিছু ভালো করা যায়?

তুমি হ্যাঁ, array যেহেতু sorted, two pointer ব্যবহার করবো — $O(1)$ extra space। L শুরুতে, R শেষে। যোগফল target-এর চেয়ে ছোট হলে যোগফল বাড়াতে হবে, তাই $L++$; বড় হলে $R--$; সমান হলে উত্তর।

ইন্টারভিউয়ার এটা কেন সঠিক? কোনো উত্তর বাদ পড়বে না তো?

তুমি যখন যোগফল ছোট, তখন $nums[L]$ -এর সাথে সবচেয়ে বড় সম্ভাব্য সংখ্যা $nums[R]$ নিয়েও ছোট হচ্ছে — তাই $nums[L]$ কোনো জোড়াতেই কাজে লাগবে না, নিরাপদে বাদ দেওয়া যায়। একইভাবে বড় হলে $nums[R]$ বাদ।

ইন্টারভিউয়ার চমৎকার। কোড লেখো।

PHP

```
function twoSumSorted(array $nums, int $target): array {
    $left = 0;
    $right = count($nums) - 1;
    while ($left < $right) {
        $sum = $nums[$left] + $nums[$right];
        if ($sum === $target) {
            return [$left, $right];
        } elseif ($sum < $target) {
            $left++; // যোগফল বাড়াতে হবে
        } else {
            $right--; // যোগফল কমাতে হবে
        }
    }
    return []; // উত্তর না থাকলে
}

echo json_encode(twoSumSorted([2, 7, 11, 15], 9)); // [0,1]
echo json_encode(twoSumSorted([-3, -1, 0, 4, 8], 3)); // [1,3]
```

তুমি ড্রাই রান: [2, 7, 11, 15], target 9। L=0, R=3: 2+15=17 > 9 → R=2। 2+11=13 > 9 → R=1। 2+7=9 → [0, 1]। ✓

তুমি Edge case: দুইটার কম element হলে লুপ চলবে না, [] রিটার্ন। নেগেটিভ সংখ্যাতেও যুক্তি একই। Complexity: time $O(n)$ কারণ প্রতি ধাপে একটা pointer এগোয়, space $O(1)$ ।

ইন্টারভিউয়ার ধন্যবাদ, ভালো হয়েছে।

এই উত্তর কেন ভালো

- কোড শুরুর আগে প্রশ্ন নিশ্চিত করেছে
- তিন স্তরে উন্নত করেছে: $O(n^2) \rightarrow O(n)/O(n) \rightarrow O(n)/O(1)$
- "কেন সঠিক" প্রশ্নের যুক্তিসহ উত্তর দিয়েছে
- নিজে থেকেই ড্রাই রান, edge case আর complexity বলেছে

অধ্যায় ৫: Mock Interview ২ — Missing Number

প্রশ্ন

0 থেকে n পর্যন্ত $n+1$ টা সংখ্যার মধ্যে n টা সংখ্যা একটা array-তে আছে (এলোমেলো)। কোন সংখ্যাটা নেই? উদাহরণ:
 $[3, 0, 1] \rightarrow 2$ (LeetCode 268)

ভূমি নিশ্চিত হই — সব সংখ্যা আলাদা, আর ঠিক একটাই নেই, তাই তো?

ইন্টারভিউয়ার হ্যাঁ।

ভূমি প্রথম উপায়: sort করে দেখি কোন index-এ $\text{nums}[i] \neq i$ — $O(n \log n)$ । দ্বিতীয়: সব সংখ্যা hash set-এ রেখে 0 থেকে n পর্যন্ত চেক করি কোনটা নেই — $O(n)$ time, $O(n)$ space।

ইন্টারভিউয়ার Space কমাতে পারবে?

ভূমি হ্যাঁ, গণিত দিয়ে। 0 থেকে n -এর যোগফল $= n(n+1)/2$ । এর থেকে array-র যোগফল বিয়োগ করলে যেটা থাকে সেটাই missing। $O(n)$ time, $O(1)$ space।

ইন্টারভিউয়ার n খুব বড় হলে সমস্যা আছে?

ভূমি অন্য ভাষায় overflow হতে পারে। তখন XOR ব্যবহার করা যায়: $0..n$ সবকিছু আর array-র সব element XOR করলে জোড়ায় জোড়ায় কেটে যায়, শুধু missing সংখ্যা থাকে। PHP-তে বড় সংখ্যা float হয়ে যায়, তাই XOR নিরাপদ।

PHP

```
// পদ্ধতি ১: গণিত (যোগফল)
function missingNumber(array $nums): int {
    $n = count($nums);
    return intval($n * ($n + 1) / 2) - array_sum($nums);
}

// পদ্ধতি ২: XOR —  $x \oplus x = 0$ ,  $x \oplus 0 = x$ 
function missingNumberXor(array $nums): int {
    $res = count($nums); // n দিয়ে শুরু
    foreach ($nums as $i => $x) {
        $res ^= $i ^ $x; // index আর মান দুটোই XOR
    }
    return $res;
}

echo missingNumber([3, 0, 1]) . " " . missingNumberXor([9, 6, 4, 2, 3, 5, 7, 0, 1]); // 2 8
```

শিক্ষা

একই প্রশ্নের ৪টা সমাধান: sort ($O(n \log n)$), hash set ($O(n)/O(n)$), গণিত ($O(n)/O(1)$), XOR ($O(n)/O(1)$, overflow-নিরাপদ)। ইন্টারভিউয়ার বারবার "আরও ভালো?" জিজ্ঞেস করলে ঘাবড়াবে না — এটাই স্বাভাবিক।

অধ্যায় ৬: Mock Interview ৩ — Maximum Subarray

প্রশ্ন

একটা integer array (নেগেটিভসহ) দেওয়া আছে। পাশাপাশি element-এর কোন অংশের (subarray) যোগফল সবচেয়ে বেশি? যোগফলটা রিটার্ন করো। উদাহরণ: $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow 6$ ($[4, -1, 2, 1]$)। (LeetCode 53)

তুমি subarray মানে পাশাপাশি, আর অন্তত একটা element নিতে হবে — ঠিক? সব নেগেটিভ হলে সবচেয়ে বড় নেগেটিভটাই উত্তর?

ইন্টারভিউয়ার হ্যাঁ, দুটোই ঠিক।

তুমি Brute force: প্রতিটা শুরু i আর শেষ j -এর যোগফল — চলমান যোগফল রাখলে $O(n^2)$ ।

ইন্টারভিউয়ার $n = 10^5$ হলে?

তুমি তাহলে $O(n^2) = 10^{10}$, খুব ধীর। একটা পর্যবেক্ষণ: প্রতিটা index-এ আমার দুইটা পছন্দ — আগের subarray-র সাথে যুক্ত হওয়া, নয়তো এখান থেকে নতুন শুরু। আগের যোগফল নেগেটিভ হলে সেটা আমাকে শুধু কমাবে, তাই ফেলে দিয়ে নতুন শুরু করি। এটা Kadane's algorithm — আসলে একটা DP: $current[i] = \max(nums[i], current[i-1] + nums[i])$ ।

ইন্টারভিউয়ার ঠিক আছে, লেখো।

PHP

```
function maxSubArray(array $nums): int {
    $current = $nums[0]; // i-তে শেষ হওয়া সেরা subarray-র যোগফল
    $best = $nums[0];
    for ($i = 1; $i < count($nums); $i++) {
        $current = max($nums[$i], $current + $nums[$i]); // নতুন শুরু, নাকি যুক্ত হবো?
        $best = max($best, $current);
    }
    return $best;
}

echo maxSubArray([-2, 1, -3, 4, -1, 2, 1, -5, 4]); // 6
echo " " . maxSubArray([-3, -1, -2]); // -1
```

i	nums[i]	current = max(nums[i], current + nums[i])	best
0	-2	-2	-2
1	1	max(1, -2) = 1 (নতুন শুরু)	1
2	-3	max(-3, -2) = -2	1
3	4	max(4, -2) = 4 (নতুন শুরু)	4
4	-1	max(-1, 4) = 3	4
5	2	max(2, 3) = 5	5
6	1	max(1, 5) = 6	6
7	-5	1	6
8	4	5	6

শিক্ষা

সব নেগেটিভ edge case আগে থেকে জিঙ্কস করায় `$best = $nums[0]` দিয়ে শুরু করার কারণ পরীক্ষার (0 দিয়ে শুরু করলে `[-3,-1,-2]`-এ ভুল উত্তর 0 আসতো)। "এটা আসলে DP" বলতে পারলে দিন ৫-এর জ্ঞান দেখানো হয়।

অধ্যায় ৭: Timed Mock Set — নিজেকে পরীক্ষা করো

টাইমার ধরো, জোরে জোরে বলো (সম্ভব হলে নিজের ভিডিও রেকর্ড করো বা বন্ধুকে ইন্টারভিউয়ার বানাও)। সমাধান দেখবে না — আটকালে সংশ্লিষ্ট দিনের PDF দেখো।

রাউন্ড	প্রশ্ন ১ (১৫ মিনিট)	প্রশ্ন ২ (২৫ মিনিট)	সাথে থিওরি প্রশ্ন
রাউন্ড ১	Valid Palindrome (125)	Group Anagrams (49)	Hash table-এ collision কী?
রাউন্ড ২	Merge Two Sorted Lists (21)	Binary Tree Level Order (102)	BFS বনাম DFS?
রাউন্ড ৩	Search Insert Position (35)	Coin Change (322)	Greedy কেন এখানে ব্যর্থ?

নিজেকে নম্বর দাও (প্রতিটা প্রশ্নে)

মাপকাঠি	হ্যাঁ হলে ১ নম্বর
কোড শুরুর আগে প্রশ্ন ও edge case নিশ্চিত করেছি	☐
Brute force বলেছি ও তার complexity	☐
Optimized সমাধান আর কেন ভালো ব্যাখ্যা করেছি	☐
কোড চলার মতো সঠিক (syntax সহ)	☐
ড্রাই রান করেছি	☐
শেষে Time ও Space বলেছি	☐

৫-৬: তুমি প্রস্তুত। ৩-৪: দুর্বল ধাপটা আরেকবার প্র্যাকটিস করো। ০-২: সংশ্লিষ্ট দিনের PDF আবার পড়ো।

অধ্যায় ৮: র‍্যাপিড-ফায়ার থিওরি (এক লাইনের উত্তর)

প্রশ্ন	উত্তর
1. Big-O কী?	ইনপুট বাড়লে সময়/মেমোরি কী হারে বাড়ে (সাধারণত worst case)।
2. $O(1)$ উদাহরণ?	Array index access, hash map lookup।
3. $O(\log n)$ উদাহরণ?	Binary search, balanced BST search।
4. $O(n \log n)$ উদাহরণ?	Merge sort, PHP sort()।
5. Space complexity কী?	অ্যালগরিদমের অতিরিক্ত মেমোরি (ইনপুট বাদে)।
6. Array access $O(1)$ কেন?	Contiguous memory: $\text{start} + \text{index} \times \text{size}$ ।
7. PHP array আসলে কী?	Ordered hash map।
8. array_shift কেন ধীর?	সব index পুনর্বিন্যাস করে, $O(n)$ ।
9. Two pointer কখন?	Sorted array, in-place, pair খোঁজা।
10. Sliding window কখন?	Contiguous subarray/substring-এর max/min/longest।
11. Binary search-এর শর্ত?	Data sorted হতে হবে।
12. mid কীভাবে নিরাপদে বের করবে?	$\text{low} + (\text{high} - \text{low}) / 2$ ।
13. Stable sort কী?	সমান মানের element-দের আগের ক্রম বজায় থাকে।
14. In-place sort কী?	$O(1)/O(\log n)$ extra space-এ sort।
15. Bubble sort-এর best case?	$O(n)$ — swapped flag দিয়ে, sorted input-এ।
16. Merge sort-এর space?	$O(n)$ ।
17. Quick sort-এর worst case?	$O(n^2)$, খারাপ pivot-এ (sorted input + শেষ element pivot)।
18. Quick sort worst case এড়াতে কীভাবে?	Random pivot / median of three।
19. Counting sort কখন?	ছোট রেঞ্জের integer; $O(n + k)$ ।
20. Hash function কী?	Key থেকে bucket index বানানোর ফাংশন।
21. Collision কী?	দুই key একই bucket-এ।
22. Collision সমাধান?	Chaining বা Open addressing।
23. Hash map worst case?	$O(n)$ — সব key এক bucket-এ।
24. Load factor?	$\text{element} \div \text{bucket}$; বেশি হলে rehash।
25. Set বনাম Map?	Set-এ শুধু ইউনিক key; Map-এ key $\rightarrow$ value।

প্রশ্ন	উত্তর
26. Linked list বনাম array?	LL: শুরুতে insert $O(1)$, access $O(n)$; Array উল্টো।
27. Doubly linked list-এর সুবিধা?	দুই দিকে চলা, জানা node $O(1)$ -এ delete।
28. Stack কী?	LIFO — push/pop top-এ।
29. Queue কী?	FIFO — পেছনে ঢেকে, সামনে বের হয়।
30. Deque কী?	দুই মাথায় $O(1)$ insert/delete।
31. Call stack কী?	ফাংশন কলের তথ্য রাখার stack।
32. Stack overflow কেন?	অতিরিক্ত গভীর/অসীম recursion।
33. Recursion-এর দুই অংশ?	Base case ও recursive case।
34. Memoization কী?	ফলাফল cache করে পুনরাবৃত্ত হিসাব এড়ানো।
35. DP-র দুই শর্ত?	Overlapping subproblems, optimal substructure।
36. Top-down বনাম Bottom-up?	Recursion+memo বনাম loop+table।
37. Greedy কী?	প্রতি ধাপে তখনকার সেরা পছন্দ, ফিরে দেখে না।
38. Greedy ব্যর্থতার উদাহরণ?	Coin change [1,3,4], amount 6।
39. Tree বনাম Graph?	Tree: cycle নেই, সংযুক্ত, $n-1$ edge, root আছে।
40. BST নিয়ম?	বামে সব ছোট, ডানে সব বড় (প্রতিটা node-এ)।
41. BST-র inorder কী দেয়?	Sorted ক্রম।
42. Skewed BST-এ search?	$O(n)$ ।
43. Tree height বনাম depth?	Height: নিচের leaf পর্যন্ত; Depth: root থেকে node পর্যন্ত।
44. BFS কী ব্যবহার করে?	Queue।
45. DFS কী ব্যবহার করে?	Stack / recursion।
46. Unweighted shortest path?	BFS।
47. Adjacency list-এর space?	$O(V + E)$ ।
48. Undirected cycle detection?	DFS + parent; visited প্রতিবেশী $\neq$ parent $\rightarrow$ cycle।
49. Bipartite কী?	দুই রঙে রং করা যায়; বিজোড় cycle নেই।
50. Connected components গণনা?	প্রতিটা unvisited node থেকে DFS/BFS, count++।

অধ্যায় ৯: PHP ইন্টারভিউ চিট শিট

কাজ	PHP কোড	খরচ
Array সাইজ	<code>count(\$a)</code>	$O(1)$
শেষে যোগ / বাদ	<code>\$a[] = \$x; / array_pop(\$a)</code>	$O(1)$
শুরুতে যোগ / বাদ	<code>array_unshift / array_shift</code>	$O(n)$ — এড়িয়ে চলো
Sort (ছোট→বড় / বড়→ছোট)	<code>sort(\$a) / rsort(\$a)</code>	$O(n \log n)$
Custom sort	<code>usort(\$a, fn(\$x, \$y) => \$x[0] <=> \$y[0]);</code>	$O(n \log n)$
Map-এ value অনুযায়ী sort	<code>asort / arsort</code> (key রাখে)	$O(n \log n)$
Key আছে?	<code>isset(\$m[\$k])</code>	$O(1)$
Default সহ গণনা	<code>\$m[\$k] = (\$m[\$k] ?? 0) + 1;</code>	$O(1)$
String দৈর্ঘ্য / অক্ষর	<code>strlen(\$s) / \$s[\$i]</code>	$O(1)$
String → অক্ষরের array	<code>str_split(\$s)</code>	$O(n)$
Array → String	<code>implode(",", \$a)</code>	$O(n)$
Sub-array	<code>array_slice(\$a, \$start, \$len)</code>	$O(len)$
নির্দিষ্ট মান দিয়ে array	<code>array_fill(0, \$n, 0)</code>	$O(n)$
Queue	<code>\$q = new SplQueue(); \$q->enqueue(); \$q->dequeue();</code>	$O(1)$
Integer ভাগ / অসীম	<code>intdiv(\$a, \$b) / PHP_INT_MAX</code>	$O(1)$
Object-এর ইউনিক id	<code>spl_object_id(\$obj)</code>	$O(1)$

PHP-এর ফাঁদ

- ফাংশনে array পাঠালে **কপি** হয় — বদলাতে `&$arr` দাও। Object (ListNode, TreeNode) reference হিসেবেই যায়।
- `==` টাইপ বদলে তুলনা করে ("1" == 1 সত্য) — `===` ব্যবহার করো।
- `in_array()` $O(n)$ — hash map-এ key রেখে `isset()` করো।
- Recursive closure-এ নিজেকে কল করতে `use (&$fn)` লাগে।
- `$a[] = $x` -এর পর array অন্য ফাংশনে গেলে কপি হয় — বড় ডেটায় সাবধান।

অধ্যায় ১০: ইন্টারভিউয়ের দিনের গাইড

আটকে গেলে কী করবে

- চুপ থেকে না। বলো: "আমি একটু চিন্তা করছি — প্রথমে brute force দিয়ে শুরু করি।"
- ছোট একটা উদাহরণ হাতে সলভ করো, নিজের ধাপগুলো মুখে বলো।
- প্যাটার্ন টেবিলটা মনে করো: sorted? → binary search/two pointer; গণনা? → hash map; পছন্দ আর "কয়টা উপায়"? → DP।
- হিন্ট চাওয়া দোষের না: "আমি X আর Y ভাবছি, আপনি কি কোনো দিক দেখাতে চান?" — হিন্ট নিয়ে সলভ করাও ভালো স্কোর পায়।

সাধারণ ভুল যা এড়াবে

ভুল	বদলে যা করবে
প্রশ্ন শোনামাত্র কোড শুরু	আগে ১-২ মিনিট প্রশ্ন ও edge case
Off-by-one (< বনাম <=)	ছোট উদাহরণে লুপের শেষ মান চেক করো
null চেক না করে ->next	Linked list/tree-তে সবসময় আগে null চেক
Graph-এ visited ভুলে যাওয়া	BFS/DFS লেখার প্রথম লাইনেই visited
Complexity না বলা	শেষে নিজে থেকেই Time ও Space বলো
একবারে পারফেক্ট কোড লেখার চেষ্টা	কাজ করা সমাধান আগে, তারপর উন্নত করো

ইন্টারভিউয়ারকে যে প্রশ্ন করতে পারো (শেষে)

- টিমে দৈনন্দিন কাজে কোন ধরনের সমস্যা বেশি সমাধান করতে হয়?
- কোড রিভিউ আর টেস্টিং কীভাবে হয়?
- নতুন জুনিয়ররা প্রথম ৩ মাসে কী শেখে / কীভাবে সাপোর্ট পায়?

শেষ মুহূর্তের চেকলিস্ট

আগের রাতে

- ☐ Top 12 কোড একবার চোখ বুলিয়ে নিয়েছি
- ☐ Complexity মাস্টার টেবিল দেখেছি
- ☐ PHP চিট শিট দেখেছি
- ☐ ৭-৮ ঘণ্টা ঘুম (সবচেয়ে গুরুত্বপূর্ণ!)

ইন্টারভিউয়ের সময়

- ☐ প্রশ্ন নিজের ভাষায় আবার বলেছি
- ☐ উদাহরণ ও edge case জিজ্ঞেস করেছি
- ☐ Brute force → optimize
- ☐ কোডের সাথে সাথে কথা বলেছি
- ☐ ড্রাই রান + complexity

শেষ কথা

তুমি ৭ দিনে complexity, array, searching, sorting, hashing, linked list, stack, queue, recursion, DP, greedy, tree আর graph কভার করেছো। Junior পজিশনে কেউ আশা করে না তুমি সব জানো — তারা দেখে তুমি পরিস্কারভাবে চিন্তা করো, কথা বলো, আর শেখার আগ্রহ আছে কি না। শুভকামনা!